Testing and debugging functions

Learn to understand, design, build, test and explain AI systems responsibly.

Project: Python readiness portfolio

THE CONNECTED WEEK

Teach revealing tests and honest regression evidence.

Supply complete candidates/cases, distinguish comparison printing from assertion stopping and assess a separate threshold.

Keep one habit: Ask why this input reveals the bug, which later checks were skipped and what earlier tests must rerun.

MAIN LESSON · SECTION 1

Prepare the connected bridge lesson

> Check prior exposure and keep execution practice distinct from paper reasoning.

Goal and main plan

Primary target: Design revealing tests for a function

Session · Sequence and minutes · Resources

> **Session:** Session 1 · Understand · 45 min

> **Sequence and minutes:** 0–7: recall and prerequisite; 7–15: inspect the concrete case and predict; 15–30: teacher models the worked trace; 30–40: learners explain a step using the text and visual; 40–45: check the core idea.
>
> **Resources:** Sections 1–6; record the brief recall in the workbook.

> **Session:** Session 2 · Practise · 45 min
>
> **Sequence and minutes:** 0–5: retrieve the procedure; 5–15: W2 completion practice; 15–23: explain and check with a partner; 23–30: each learner answers the misconception check; 30–40: targeted reteaching or a harder variation; 40–45: prepare the investigation.
>
> **Resources:** Sections 7–10; use the matching workbook W2 and misconception check.

> **Session:** Session 3 · Investigate and transfer · 45 min
>
> **Sequence and minutes:** 0–5: retrieve the decision rule; 5–20: investigate or trace; 20–30: start the independent case; 30–38: connect the project; 38–43: act on feedback; 43–45: plan the return check.
>
> **Resources:** Use “Run the investigation”, “Connect your project”, “Show what you know”, “Study a complete case” and “Try a new case independently”. Give extra time for the full task set. If this week has a coding lab, use its named section; section numbers can change when extra explanations are included.

This is a starting allocation of 135 minutes, not a validated duration. Preserve all named topics. Schedule extra time for connected examples, the full investigation, independent cases and project building.

Repeated copies of a case share one recorded attempt; they are not new **evidence**.

Case exposure check: this source example appears in ai-2/6, python-bridge/12. Reuse a familiar case for recall or modelling; verify that the case used to assess transfer has not already received feedback.

Retrieval and prerequisite guidance

Reading structured data

Without opening the old lesson, explain one key idea from Reading structured data. Give an example and a mistake that the idea helps you avoid.

Look for: **Process** a list of records consistently

Reference example, not the only acceptable wording: There are 2 records. The second has 0 spare spaces.

Return values and local state

Without opening the old lesson, explain one key idea from Return values and local state. Give an example and a mistake that the idea helps you avoid.

Look for: Use a returned value in a later calculation

Reference example, not the only acceptable wording: result is 9 and doubled is 18. Nothing is displayed by these assignments or the returning function. print(doubled) would display 18.

Loops and trace tables

Without opening the old lesson, explain one key idea from Loops and trace tables. Give an example and a mistake that the idea helps you avoid.

Look for: Trace variable values before and after each loop iteration

Reference example, not the only acceptable wording: 2, 3 and 4; the stop value is excluded. The body runs three times.

Teach Week 12 as a two-session investigation using fictional data. A function can pass one example by coincidence.

Start with a **written rule**, calculate expected results independently, and include ordinary, zero and **boundary** cases. We will test a subtraction function under the contract 0 <= used <= capacity.

Assess the three stated outcomes through the independent task, not by copying the worked result.

Materials and readiness

Python 3 editor with a fresh local practice file

Paper trace table and the student workbook

Fictional **input** records from this lesson; no real student identifiers

7.

End goal

Trace testing and debugging functions and explain the changed program before running it.

<details>
<summary>Earlier wording, alternatives and picture transcripts</summary>

Teach the learning cycle

Retrieval answer guidance

Outcomes

Design revealing tests for a function

Compare expected and actual results

Repair a defect and rerun earlier cases

Preparation

Prerequisite answer

End goal for the pictorial case

This added worked case illustrates the week's main idea. Retain all original and connected topics; schedule its practice within the teaching cycle.
</details>

MAIN LESSON · SECTION 2

Teach a test as a contract-to-result comparison

> Keep the 8 suite,6 suite, tiny 3 boundary and selected 10 boundary separately labelled.

Read independent expectations and assertion semantics

A function can pass one example by coincidence. Start with a **written rule**, calculate expected results independently, and include ordinary, zero and **boundary** cases.

We will test a subtraction function under the contract 0 <= used <= capacity.

Each parenthesized group is a tuple, an ordered group of values. The loop unpacks each group into three names. Equality compares the actual result to the independently calculated expected result.

The worked script prints True three times. An assert actual == expected could instead stop at the first mismatch with **Assertion**Error.

Supply the complete W2 candidate and cases

```
def remaining(capacity, used):
    return 4  # intentional candidate defect: ignores both arguments

cases = [(6, 2, 4), (6, 6, 0), (6, 0, 6)]
for capacity, used, expected in cases:
    actual = remaining(capacity, used)
    print(actual, expected, actual == expected)
```

Read the pictured tiny contract

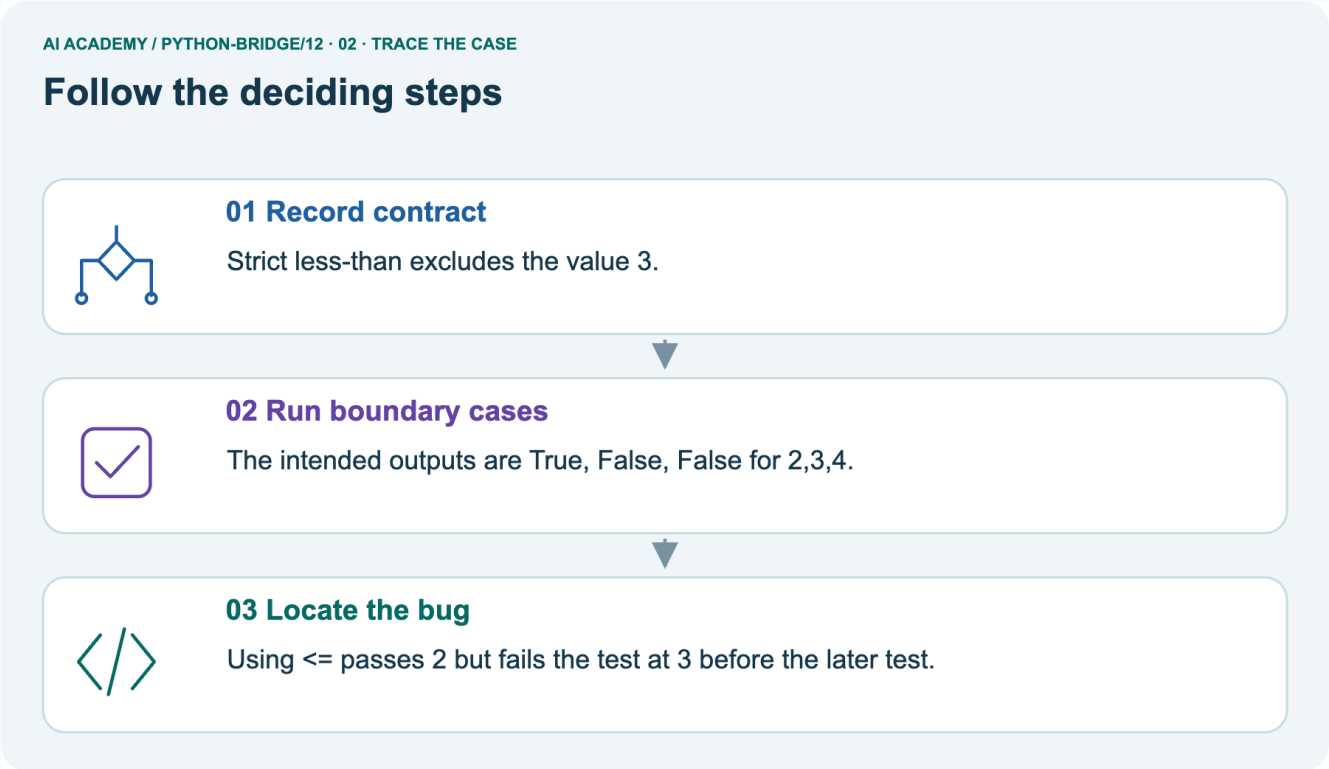
Use a test that catches the **boundary** bug. A function should classify an integer as tiny exactly when it is less than three.

Python 3 intended function: return x<3. **Test case**s: (2,True),(3,False),(4,False). Candidate bug: return x<=3.

The complete tiny program below uses assertions in normal test execution. Correct <3 passes 2,3,4 and prints the completion line. Changing only < to <= passes the first assertion, fails at 3 with AssertionError and never runs the assertion for 4 or final print. The graphic compares expression results one at a time; it is not a trace claiming later assertions executed after failure.

The completion message is a literal string, not an automatic count of executed checks. If removing a test, relabel it for the two retained checks. These are local test checks, not a production input-validation mechanism.

Trace the strict-boundary defect



Follow the labelled picture with the matching explanation. Open the vector diagram

Read the picture one step at a time

STEP	WHAT HAPPENS AND WHY
1. Record contract	Strict less-than excludes the value 3.
2. Run boundary cases	The intended outputs are True, False, False for 2,3,4.
3. Locate the bug	Using <= passes 2 but fails the test at 3 before the later test.

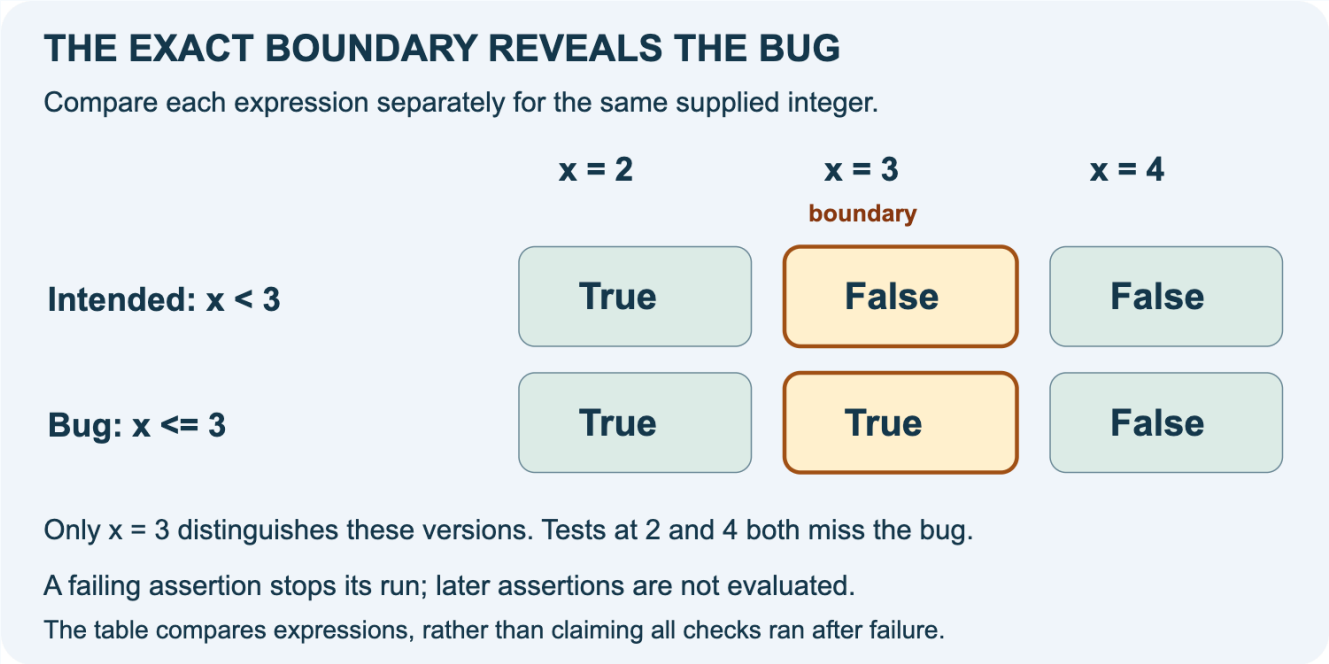
Match each pictured step to the supplied facts. This is a worked teaching case, not an independent assessment.

A revealing test distinguishes a correct implementation from a plausible mistake. Checking only values far from the **boundary** would miss this inclusive-versus-strict bug.

```
def tiny(x):
    return x < 3

assert tiny(2) is True
assert tiny(3) is False
assert tiny(4) is False
print("three checks passed")
```

Inspect the expression comparison graphic



Predict removing only the revealing test

Change: Remove the test at 3 and keep only 2 and 4.

Prediction: The buggy <= function passes those remaining tests.

Reason: Both procedures agree away from the equality **boundary**.

State bounded-suite and input-type limits

Passing these cases does not prove correctness for every type or **input**; the contract and coverage determine what was actually checked.

Cover the result. Ask learners to name the **input**s and predict the next step. Uncover one step, connect the icon to the actual procedure, and explain its **evidence**. Then change the supplied condition and compare.

The picture is a representation; it does not document execution of a real AI model.

▼ Additional guided key · Constant 5 on capacity 8

It passes (8, 3) but fails (8, 8), returning 5 instead of 0. Exit: No. Negative counts are outside our nonnegative-**input** contract and were not tested; rejection behavior must first be defined.

▼ Earlier wording, alternatives and picture transcripts

Term	Meaning
Test case	Inputs and an expected result
Boundary	An edge of a permitted range
Regression	A previously working case that fails after a change
Assertion	A check that raises an error when its condition is false

Follow the evidence

STEP OR CASE	EVIDENCE	MEANING
Ordinary case	8 and 3	Expect 5
Full case	8 and 8	Expect 0
Unused case	8 and 0	Expect 8

Read each row from left to right and connect the evidence to the stated rule.

Model the reasoning aloud

Use the complete worked example below before asking learners to complete W2. Reveal a small part, invite a **prediction**, then explain the deciding step.

These teacher notes contain solutions; do not reveal the independent case answer during modelling.

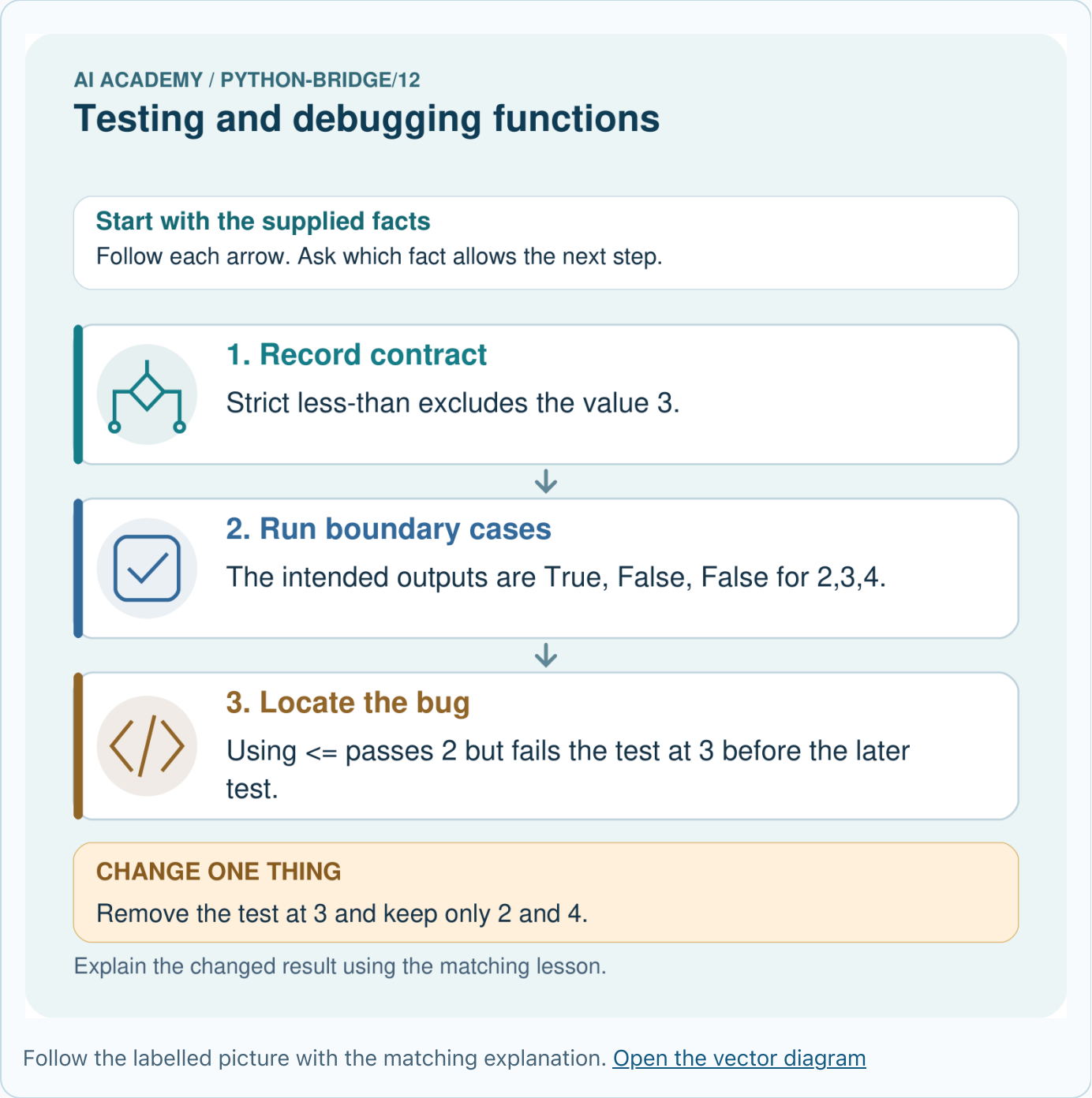
> **Would replacing the body with return 5 pass every worked case? Identify the first revealing test.**

Thinking move	Teacher prompt
Plan	What is given? What is the question? Which procedure fits these facts?
Do	Point to each input and intermediate step in the worked trace. Name the rule that allows the next step.
Check	Does the output answer the question? Which assumption or limitation prevents a larger claim?

Use the diagram and verbal explanation together. Ask learners to match a sentence to a specific visual step; do not assign learners fixed visual or auditory learning styles.

A pictorial worked case: Testing and debugging functions

The facts we will use



Follow the labelled picture with the matching explanation. Open the vector diagram

Why this result follows

What this example does not establish

Change one thing in the pictured case

Ask while showing the picture

MAIN LESSON · SECTION 3

Guide the trace, then fade the help

> Collect predictions before execution or feedback.

Main completion task and key

Completion task: A faulty function always returns 4. Which of your three cases pass and fail?

Teacher check: Only (6, 2) passes. The full and unused cases fail.

Start by identifying the given **input**s together. Leave the intermediate steps blank. If needed, model one step, then withdraw that help on the next step. Move to a full trace only when the partial trace is understood.

For partners, alternate explainer and checker; collect a response from every learner before discussion.

Individual misconception check

Use the worked case for Testing and debugging functions. Proposed explanation: “A successful run proves the conclusion”. Decide whether this explanation is supported.

Point to the step or supplied fact that decides; explain your repair if needed.

Expected correction: Passing this small suite supports these particular cases. It neither proves every allowed **input** correct nor checks invalid **input** handling.

A test outside the contract needs a separately specified expected behavior.

Collect explanations, not only yes/no votes. If a misunderstanding is common, re-model the relevant step for the class.

If it affects a few learners, give a short targeted group explanation while others solve a more demanding variation.

Pictorial question and changed-case key

Predict the three correct **output**s. Which test first catches <=, and why do tests at only two and four miss the bug?

Expected **output**s are True, False, False. The equality **input** 3 is the first failure for <=. At 2 and 4 the strict and inclusive comparisons agree.

Changed-case reasoning: The buggy <= function passes those remaining tests. Both procedures agree away from the equality **boundary**.

Collect each learner's explanation before discussion. Use the first missing connection between **input**, deciding step and result to choose support. This question is worked-case practice, not fresh mastery **evidence**.

▼ Support and alternative plan

Time or stage	Teacher action
Session 1 start to 7 minutes	Retrieve the prerequisite and explain the purpose in the opening section.
7 to 20 minutes	Read the worked example and diagram; ask learners to trace each step aloud.
20 to 33 minutes	Complete W1 and W2, then compare predictions with observed results.
33 to 45 minutes	Collect the guided prediction before running; discuss a mismatch and its cause.
Session 2 start to 8 minutes	Retrieve the main rule without showing the previous answer.
8 to 20 minutes	Use W3 to diagnose and repair the stated failure; rerun an earlier case.
20 to 35 minutes	Collect W4 independently, including code or plan, predictions and explanations, before hints.
35 to 45 minutes	Discuss W5 and the exit; use the fresh retry when support was needed.

Use the worked example in Sections 3 and 6. Collect a **prediction** before revealing its result; ask the learner to point to each intermediate fact.

Provide a blank **input**s expected actual table. Extension: use assert for the same cases and explain why only the first failure is reported.

▼ Earlier wording, alternatives and picture transcripts

Compact alternative: this two-session sequence covers selected core tasks. Use the three-session weekly plan for the full lesson, independent transfer, feedback and project work; allow extra time for extensions.

Reduce help deliberately

Individual reasoning check

Pictorial practice question

Reasoned teacher answer

MAIN LESSON · SECTION 4

Run the complete supplied investigation

> Preserve expected, observed and paper-trace records.

Supplied case materials and checked guidance

W2 constant 4 actuals 4,4,4 and expecteds 4,0,6; repair subtraction and rerun all. Main faulty addition 11,16,8 versus 5,0,8, so one accidental pass. Tiny <= fails assertion 3, later 4 and print skipped; remove 3 and 2/4 both pass without repair. Selected below_limit prints completion with correct <10; <=10 fails assertion for 10 before final print. Rerun 9,10,11 after repair; no universal/type-safety claim.

Earlier stock task retains its own inputs and expected outputs.

Diagnose misconceptions

A successful run proves the conclusion: Passing this small suite supports these particular cases. It neither proves every allowed **input** correct nor checks invalid **input** handling.

A test outside the contract needs a separately specified expected behavior.

Matching the example is enough: Require the new W4 case, explicit **prediction**s and an explanation. Fresh retry: bundles * 3 + singles with (3, 1), (0, 1), (3, 0) expects 10, 1, 9. A constant return 10 passes only the first.

A convenient shortcut preserves meaning: A faulty return capacity + used gives 11, 16 and 8. The unused case passes even though the rule is wrong. Repair the operator to subtraction and rerun all three cases.

Never change an expected result merely to match faulty **output**.

Use Section 9 for the **evidence** ledger. Add a **boundary** or missing-**input** case and record the expected response before checking. More successful repeats of one case do not establish wider coverage.

Choose accessible support

What the response shows	Next teaching action
Missing or misread facts	Read the case aloud, mark supplied versus unknown facts, and let the learner restate it.
Wrong procedure	Return to the deciding step in the worked case. Contrast it with the proposed incorrect explanation.
Arithmetic or implementation error	Trace intermediate values or lines; preserve the first mismatch and retry that step.

What the response shows	Next teaching action
Correct result without a reason	Ask for a trace and a changed case before marking independent understanding.
An unsupported wider claim	Ask which supplied evidence supports the claim and which further observation would be needed.

Offer an oral explanation, drawing, enlarged visual or fewer examples when that removes a reading/writing barrier. Keep the same reasoning goal.

Stretch secure learners by changing a **boundary** or assumption and asking them to defend the effect, not simply by assigning more of the same questions.

▼ **Earlier wording, alternatives and picture transcripts**

Match the support to the first error

MAIN LESSON · SECTION 5

Assess the selected bridge transfer task

> If a learner has seen this exact case, use a fresh parallel input and record exposure/support.

Selected independent task and checked key

Selected case: Predict the **output**. If the function mistakenly uses x<=10, identify the first failing **assertion** and whether the final print happens. State the repair and which earlier tests must be rerun.

Do these cases prove all **input** types are handled?

```
def below_limit(x):
    return x < 10
assert below_limit(9) is True
assert below_limit(10) is False
```

```
assert below_limit(11) is False
print("three checks passed")
```

The supplied function prints three checks passed. With <=10, the **assertion** for10 fails first and the final print does not run.

Repair the comparison to<10 and rerun all three cases to check the fix and preserve neighbouring results. These numerical **boundary** tests do not establish every value or **input** type is safe.

Judge trace and support

Dimension	Evidence required
Explanation	Names the applicable idea or procedure and ties it to the supplied facts.
Trace and result	Preserves the given inputs and shows the relevant intermediate steps; distinguishes a prediction from an observed execution.
Limits	States the assumptions, missing evidence or conditions under which the result would change.

Record each dimension as not yet, with support, or independent. Accept a valid alternative procedure with a defensible trace.

Do not penalise an honestly recorded model failure if the learner correctly explains its **evidence** and limits.

Give one repair action

Name the earliest unsupported step and give one action to repair it. Have the learner make that repair now and explain why. Then assess the relevant skill on a fresh, fully specified case.

Confirm it was not previously practised with feedback; solve and check the new case before assigning it. A follow-up design prompt is a starting brief, not automatically an equivalent test.

W1 Write expectations

Give expected results for remaining(6, 2), remaining(6, 6) and remaining(6, 0).

4, 0 and 6.

W2 Find a revealing case

A faulty function always returns 4. Which of your three cases pass and fail?

Only (6, 2) passes. The full and unused cases fail.

W3 Repair and retest

A faulty function adds capacity and used. For (6, 2), state actual and expected. Repair it and name a **regression** case.

Actual 8, expected 4. Subtract used; rerun a prior case such as (6, 0), expecting 6.

W5 Limit the conclusion

All three stock tests pass. Does that prove correct handling of negative counts?

No. Negative counts are outside our nonnegative-**input** contract and were not tested; rejection behavior must first be defined.

▼ Earlier wording, alternatives and picture transcripts

Guided and exit questions

It passes (8, 3) but fails (8, 8), returning 5 instead of 0. Exit: No. Negative counts are outside our nonnegative-**input** contract and were not tested; rejection behavior must first be defined.

W1 Write expectations

W2 Find a revealing case

W3 Repair and retest

W4 Test a new function independently

Define stock(packs, loose) to return packs * 4 + loose. Create expected tests for (2, 3), (0, 3), (2, 0), then show which detects a faulty packs + loose body.

Expected 11, 3, 8. Faulty results 5, 3, 2; first and third reveal it. Use independent expectations in the test loop.

W5 Limit the conclusion

Independent assessment and fresh retry

Independent W4 **evidence**: Expected 11, 3, 8. Faulty results 5, 3, 2; first and third reveal it. Use independent expectations in the test loop. Record whether the learner needed no help, a hint or explanation.

Fresh retry: bundles * 3 + singles with (3, 1), (0, 1), (3, 0) expects 10, 1, 9. A constant return 10 passes only the first.

Assess independent transfer

Selected case answer

Feedback that the learner uses

MAIN LESSON · SECTION 6

Keep the portfolio, support and delayed check

> Track code execution separately from a correct paper trace.

Project evidence

Add a labelled testing and debugging functions **evidence** sheet to Python readiness portfolio. Show how you can design revealing tests for a function.

Use the supplied fictional case and keep its **input**s, intermediate steps, calculation or **prediction**, and limitation. Connect this record to last week's record; a matching answer without a trace is insufficient.

For Python readiness before the Responsible Plant Classifier, retain a trace of the values, the predicted **output**s and the changed-case explanation. Use actual Python 3 results to check the **prediction** after reasoning first.

Support and selected assessment guidance

Provide a blank **input**s expected actual table. Extension: use assert for the same cases and explain why only the first failure is reported.

Selected independent answer: The supplied function prints three checks passed. With <=10, the assertion for10 fails first and the final print does not run. Repair the comparison to<10 and rerun all three cases to check the fix and preserve neighbouring results. These numerical boundary tests do not establish every value or input type is safe.

Keep earlier W4 transfer as additional practice; do not substitute its numbers or key for this selected task. Record prior exposure and use a fresh parallel case after support.

Delayed return

Suggested return points in this level: plan a review after the level ends. Use actual classroom dates, adjusting for holidays and gaps. One, three and six teaching weeks are scheduling choices, not a claimed optimal spacing.

At a delayed check, ask for the idea from memory and a changed application. Compare with the earlier independent response and record support used.

For a new level, revisit the previous capstone before checking the new prerequisites.

Record	Teacher evidence
Learner code and date	
Concept and case used; prior exposure	
Explanation / trace / limits	
Support and first repair	
Changed case and delayed outcome	
Next teaching action	

Use the **evidence** to choose reteaching or the next step. A completed page, confidence rating or attractive project does not by itself demonstrate understanding.

Bridge to the next topic

Next week: Rows columns and schemas. Reuse this week's explicit contracts and **evidence**.

Retain the project ledger and assess a changed case after support. Check the relevant prerequisite before carrying this component forward.

▼ Earlier wording, alternatives and picture transcripts

Plan the delayed return

Project evidence from this case

Reference bank and extra practice

Choose the reference that helps your next step.

▼ **Reference · Alternative planning checklist**

REFERENCE · SECTION 7

Reference · Alternative planning checklist

> Use alongside the main route without repeating the teaching cycle.

Planning and assessment reference

Teaching target: Design revealing tests for a function

Prepare the labelled worked-case picture, the matching workbook, fictional **input** cards and a place for individual responses. Check a local Python 3 environment before class; have a paper-trace version ready.

Session · Plan · Resources

> **Session:** Session 1 · Understand · 45 min
>
> **Plan:** 0–7: recall and prerequisite; 7–15: inspect the concrete case and predict; 15–30: teacher models the worked trace; 30–40: learners explain a step using the text and visual; 40–45: check the core idea.
>
> **Resources:** Sections 1–6; record the brief recall in the workbook.

> **Session:** Session 2 · Practise · 45 min
>
> **Plan:** 0–5: retrieve the procedure; 5–15: W2 completion practice; 15–23: explain and check with a partner; 23–30: each learner answers the misconception check; 30–40: targeted reteaching or a harder variation; 40–45: prepare the investigation.

> **Resources:** Sections 7–10; use the matching workbook W2 and misconception check.

> **Session:** Session 3 · Investigate and transfer · 45 min
>
> **Plan:** 0–5: retrieve the decision rule; 5–20: investigate or trace; 20–30: start the independent case; 30–38: connect the project; 38–43: act on feedback; 43–45: plan the return check.
>
> **Resources:** Use “Run the investigation”, “Connect your project”, “Show what you know”, “Study a complete case” and “Try a new case independently”. Give extra time for the full task set. If this week has a coding lab, use its named section; section numbers can change when extra explanations are included.

Before class, solve the supported practice: A faulty function always returns 4. Which of your three cases pass and fail?

Reasoned practice answer: Only (6, 2) passes. The full and unused cases fail.

Check every learner before discussion: Use the worked case for Testing and debugging functions. Proposed explanation: “A successful run proves the conclusion”. Decide whether this explanation is supported.

Point to the step or supplied fact that decides; explain your repair if needed.

Misconception repair: Passing this small suite supports these particular cases. It neither proves every allowed **input** correct nor checks invalid **input** handling.

A test outside the contract needs a separately specified expected behavior.

Support: read the exact case aloud, enlarge the labelled picture, supply **input** cards and model only the first deciding step. Accept a spoken explanation or drawing; retain the reasoning goal.

Extension: Remove the test at 3 and keep only 2 and 4. Require a **prediction** and a reason before comparing. Expected result: The buggy <= function passes those remaining tests.

Reason: Both procedures agree away from the equality **boundary**.

Independent assessment: withhold the reserved answer until the learner has attempted the new case. Record support separately from correctness.

Criterion	Not yet	With support	Independent
Explanation	Names an answer without a deciding fact	Uses a hint to link fact and decision	Explains the deciding fact in own words
Trace or test	Cannot connect input and result	Completes steps with prompts	Traces or tests a changed case accurately
Limits and revision	Claims more than evidence supports	Names a limit after a prompt	States a defensible limit and useful next test

Project checkpoint: Python readiness portfolio. Keep runnable code, **prediction**s, actual **output**s, one repaired error and an explanation of a function on changed **input**s.

Use the weekly teacher answer for topic-specific reasoning. A saved progress tick is an activity record; judge mastery from the learner's explanation and **evidence**.

▼ **Reference · Feedback and evidence records**

REFERENCE · SECTION 8

Reference · Feedback and evidence records

> Keep individual first attempts separate from supported rehearsal.

Feedback and repair

Use the two formative questions as supported practice, not as independent mastery scores. Discuss the deciding **evidence** rather than accepting a guessed selection.

The feedback comes from the reviewed public worked case. Do not reveal the separately reserved independent teacher answer until the learner has attempted that case.

Ask students to predict before running code. Preserve errors and actual **output**s; collect a changed test and an explanation. The branch-**boundary** and indexing lessons deliberately include failure cases.

Use the earlier worked-explanation links to address a specific missing building block, then return to the current question. Adapt this suggestion after observing the learner; it is not an automatic diagnosis.

Collect the completed-work portfolio as **evidence**. Record support separately from correctness, and assess explanation, trace/test and limitations. Progress ticks and quiz selections alone do not establish mastery.

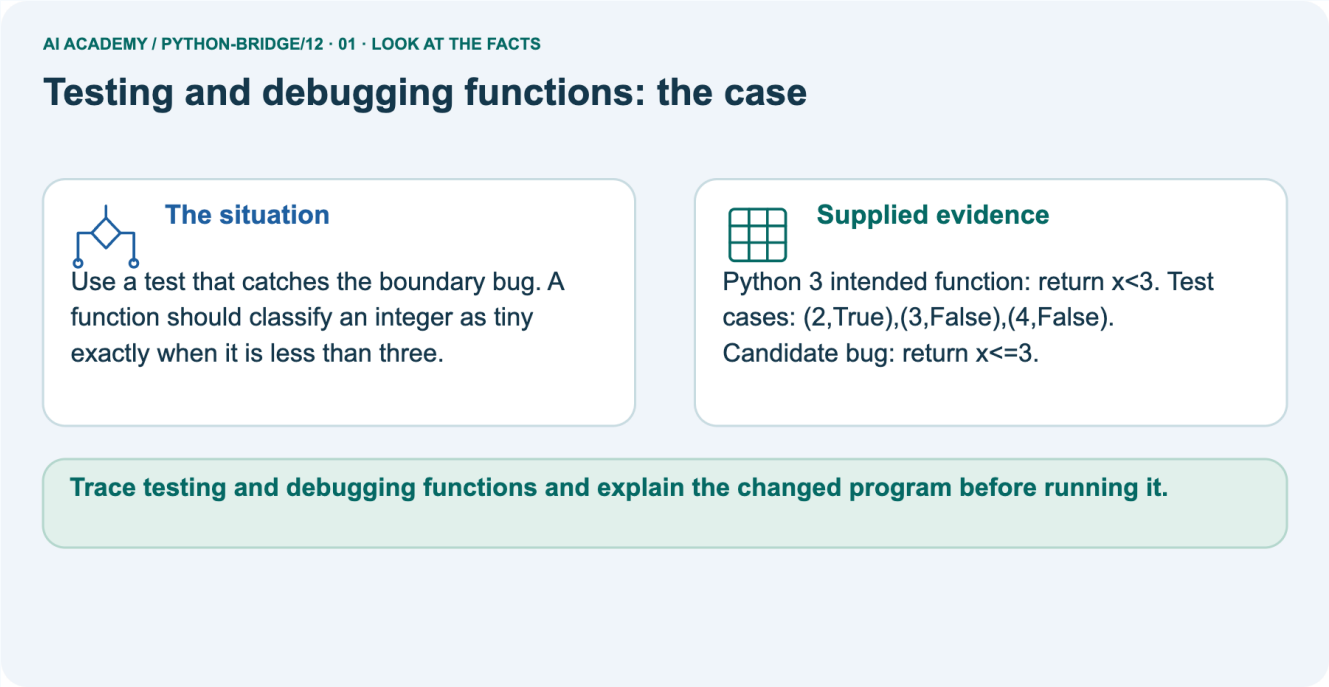
▼ **Reference · Teach with the six pictures**

REFERENCE · SECTION 9

Reference · Teach with the six pictures

> Use the relevant picture for a specific reasoning move.

01. Testing and debugging functions: the case



Follow the labelled picture with the matching explanation. Open the vector diagram

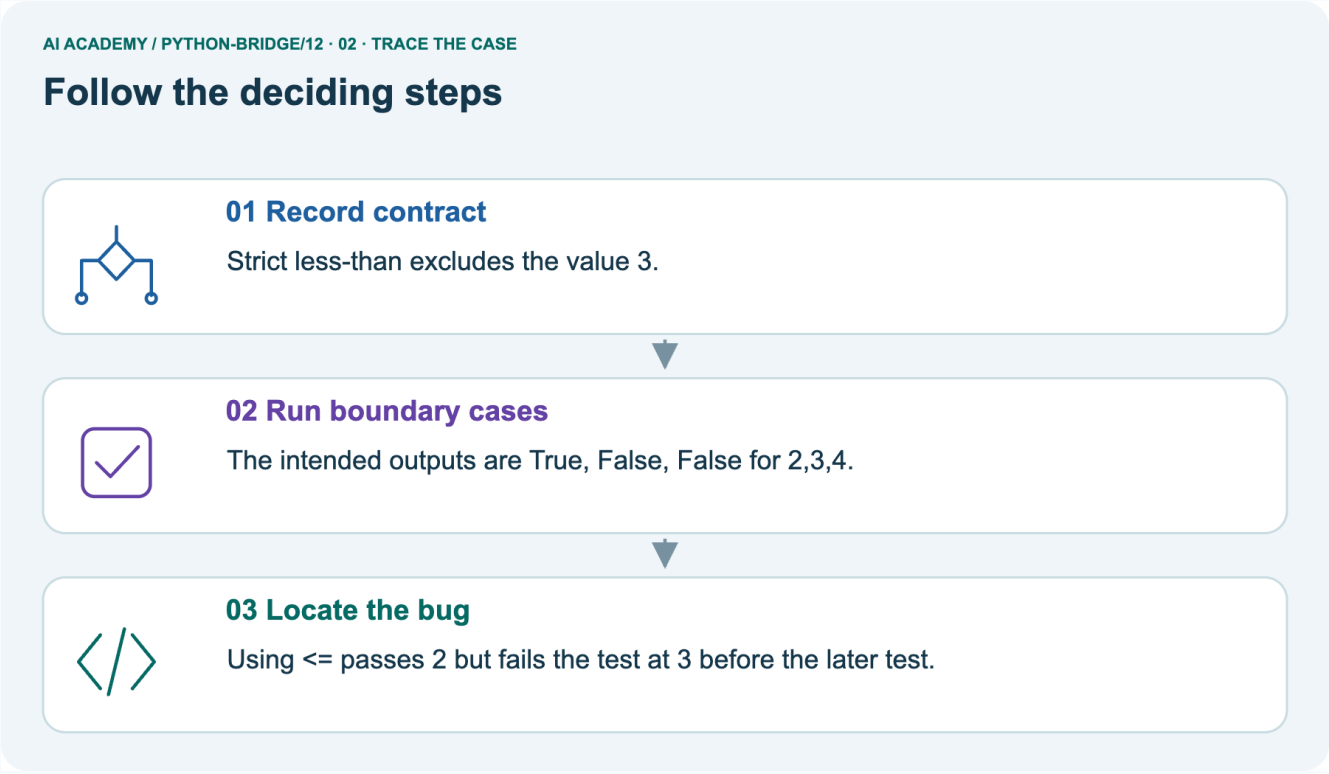
Use a test that catches the **boundary** bug. A function should classify an integer as tiny exactly when it is less than three. Supplied facts: Python 3 intended function: return x<3. **Test case**s: (2,True),(3,False),(4,False).

Candidate bug: return x<=3.

Point to the given fact that matters most. Explain why it matters.

Teacher prompt: Point to the given fact that matters most. Explain why it matters. Ask learners to describe one label or arrow in ordinary words. Use the adjacent transcript if the image is difficult to see.

02. Follow the deciding steps



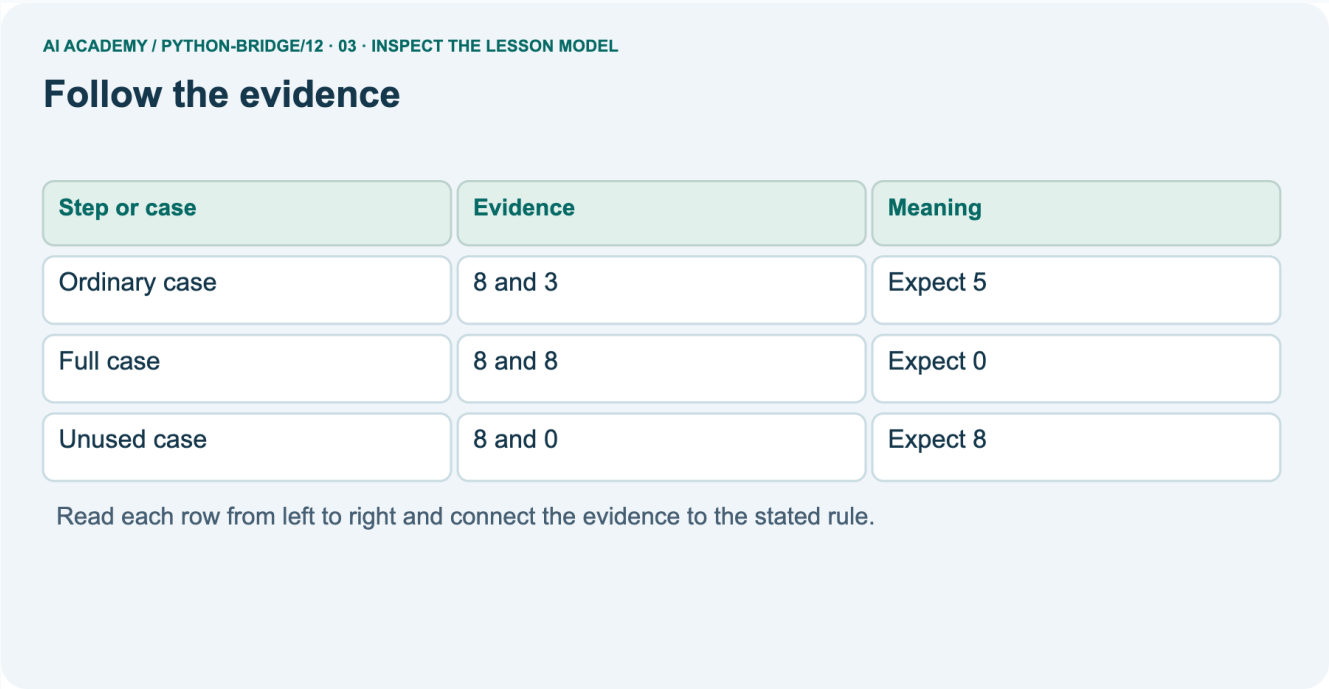
Follow the labelled picture with the matching explanation. Open the vector diagram

Record contract: Strict less-than excludes the value 3. → Run **boundary** cases: The intended **output**s are True, False, False for 2,3,4. → Locate the bug: Using <= passes 2 but fails the test at 3 before the later test.

Point to each numbered stage and explain which recorded **input** it uses.

Teacher prompt: Point to each numbered stage and explain which recorded **input** it uses. Ask learners to describe one label or arrow in ordinary words. Use the adjacent transcript if the image is difficult to see.

03. Follow the evidence



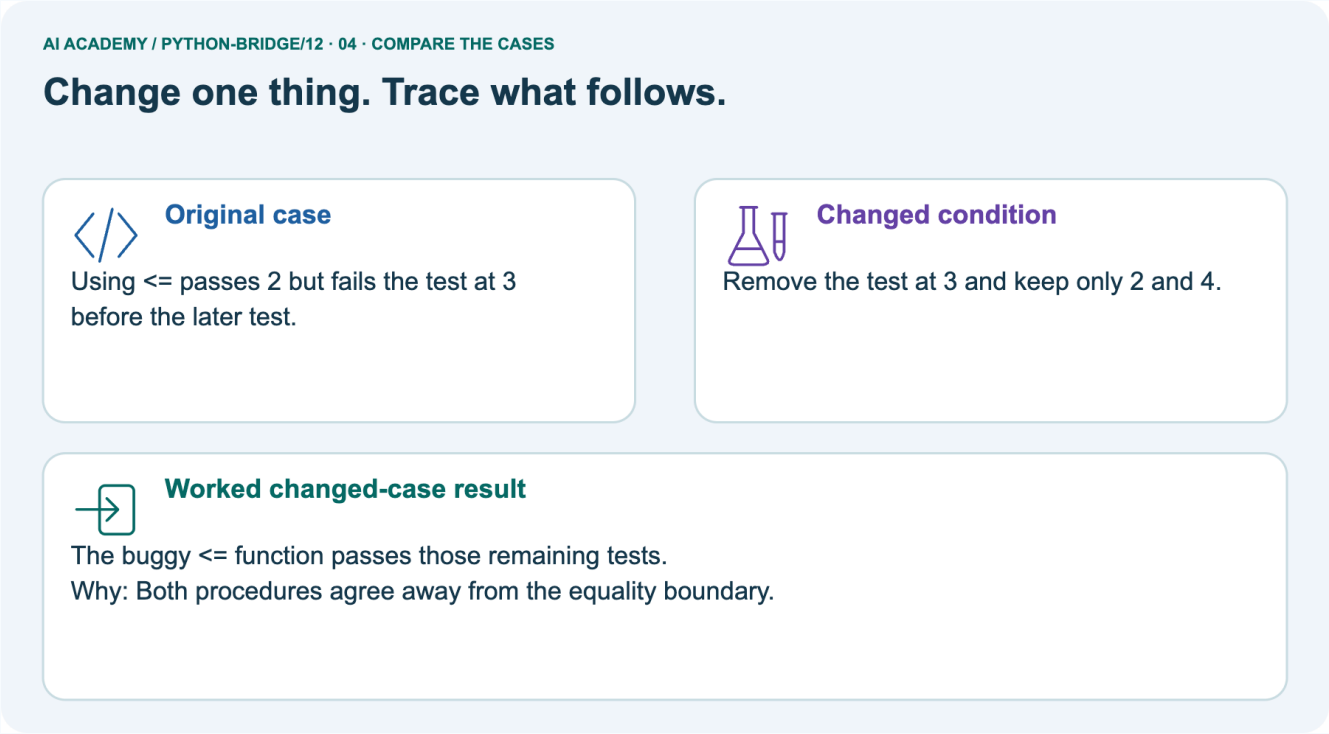
Follow the labelled picture with the matching explanation. Open the vector diagram

Follow the **evidence**. Ordinary case | 8 and 3 | Expect 5. Full case | 8 and 8 | Expect 0. Unused case | 8 and 0 | Expect 8

Read the labels and values. Explain the relationship the model shows.

Teacher prompt: Read the labels and values. Explain the relationship the model shows. Ask learners to describe one label or arrow in ordinary words. Use the adjacent transcript if the image is difficult to see.

04. Change one thing. Trace what follows.



Follow the labelled picture with the matching explanation. Open the vector diagram

Original case: Using <= passes 2 but fails the test at 3 before the later test. Changed condition: Remove the test at 3 and keep only 2 and 4. Expected result: The buggy <= function passes those remaining tests.

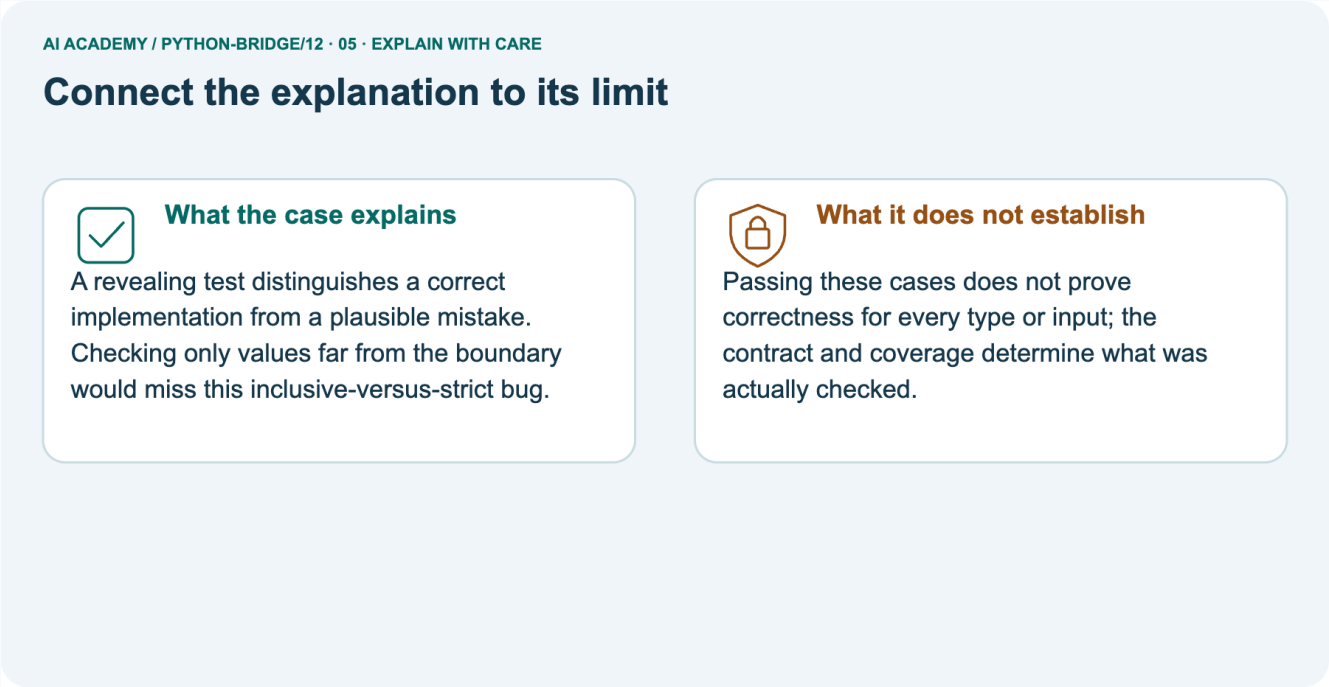
Reason: Both procedures agree away from the equality **boundary**.

Predict before checking the worked changed-case result. Name what changed and what the supplied facts allow you to conclude.

Teacher prompt: Predict before checking the worked changed-case result. Name what changed and what the supplied facts allow you to conclude. Ask learners to describe one label or arrow in ordinary words.

Use the adjacent transcript if the image is difficult to see. Changed-case reasoning: Both procedures agree away from the equality **boundary**.

05. Connect the explanation to its limit



Follow the labelled picture with the matching explanation. Open the vector diagram

A revealing test distinguishes a correct implementation from a plausible mistake. Checking only values far from the **boundary** would miss this inclusive-versus-strict bug.

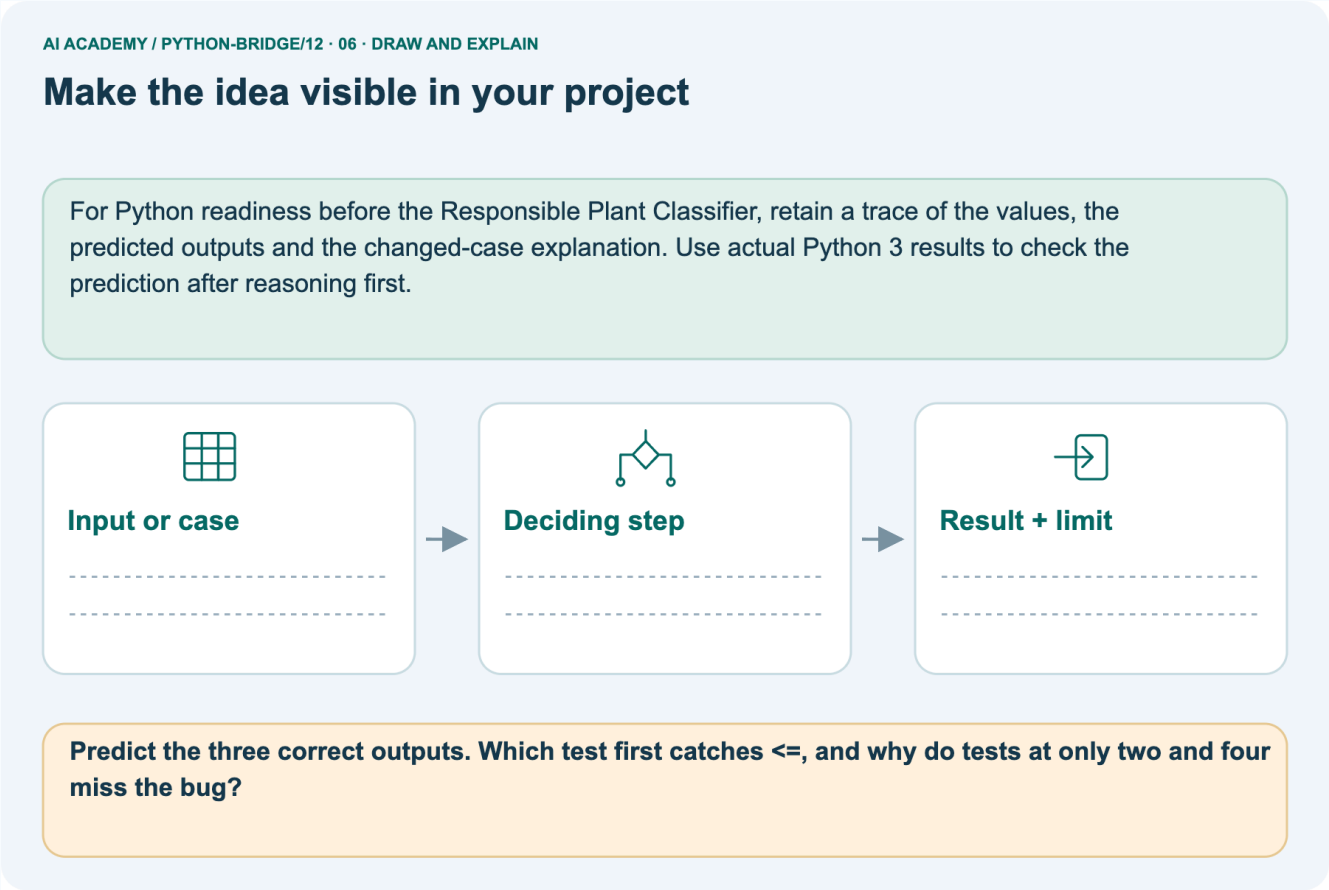
Limit: Passing these cases does not prove correctness for every type or **input**; the contract and coverage determine what was actually checked.

Explain the case, then point to a limit on the claim.

Teacher prompt: Explain the case, then point to a limit on the claim. Ask learners to describe one label or arrow in ordinary words. Use the adjacent transcript if the image is difficult to see.

Keep the stated scope: Passing these cases does not prove correctness for every type or **input**; the contract and coverage determine what was actually checked.

06. Make the idea visible in your project



Follow the labelled picture with the matching explanation. Open the vector diagram

For Python readiness before the Responsible Plant Classifier, retain a trace of the values, the predicted **output**s and the changed-case explanation. Use actual Python 3 results to check the **prediction** after reasoning first.

Draw your **input**, deciding step, result and limit. Supported question: Predict the three correct **output**s. Which test first catches <=, and why do tests at only two and four miss the bug?

Draw your project trace and justify the deciding step using **evidence**.

Teacher prompt: Draw your project trace and justify the deciding step using **evidence**. Ask learners to describe one label or arrow in ordinary words. Use the adjacent transcript if the image is difficult to see.

Supported practice key: Expected **output**s are True, False, False. The equality **input** 3 is the first failure for <=. At 2 and 4 the strict and inclusive comparisons agree.

▼ Earlier wording, alternatives and picture transcripts

Use one picture at a time. Hide a worked result before asking for a **prediction**, then reveal it for feedback. These are supported teaching cases; retain the separate independent assessment for a fresh first attempt.

01. Testing and debugging functions: the case

02. Follow the deciding steps

03. Follow the evidence

04. Change one thing. Trace what follows.

05. Connect the explanation to its limit

06. Make the idea visible in your project

▼ Optional interactive practice and completed work

Save or print your responses for your portfolio. Progress and drafts on the hosted website stay in the current browser. Use only fictional or teacher-approved data.